Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Departamentul Ingineria Software și Automatică
Program: Software Engineering

Operating Systems

Laboratory Work No. 7

Three Small Programs: Processes, Redirection, a Pipe

Track A: The Real OS, from the Outside (Linux)

Done by: st. gr. FAF-241, Ermicev Sofia

Checked by: Patricia Reițman

Chișinău – 2026

Contents

1 Purpose of the Work **2**

2 Part 1. fork + exec + wait **2**

2.1 Observe answers 2

3 Part 2. A Zombie, on Purpose **3**

3.1 Observe answers 3

4 Part 3. Redirection: ls -l > out.txt **3**

4.1 Observe answers 4

5 Part 4. A Pipe: ls /etc | wc -l **4**

5.1 Observe answers 5

6 Conclusion **5**

1. Purpose of the Work

The purpose of this laboratory work is to reproduce basic shell mechanisms directly in C: creating a process with `fork()`, replacing it with `exec()`, waiting for it, observing a zombie, redirecting standard output with file descriptors, and connecting two processes through a pipe.

2. Part 1. fork + exec + wait

```
$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 41031
child: my PID is 41032, my parent is 41031
total 24
-rwxr-xr-x 1 sofia sofia 16400 Oct 19 14:10 run
-rw-r--r-- 1 sofia sofia   919 Oct 19 14:09 run.c
parent: child 41032 ended with status 0
```

Figure 1.1 – Actual execution of run with ls -l

```
parent: my PID is 41555
child: my PID is 41556, my parent is 41555
Mon Oct 19 14:11:58 EEST 2026
parent: child 41556 ended with status 0
```

Figure 1.2 – Actual execution after changing exec to date

```
parent: my PID is 42078
child: my PID is 42079, my parent is 42078
exec: No such file or directory
parent: child 42079 ended with status 127
```

Figure 1.3 – Actual failed exec experiment

2.1. Observe answers

1. `perror("exec")` runs only after an unsuccessful `execlp()`, because successful `exec` replaces the old process image and does not return.

2. The child reported the PID of its actual parent; for example PID 41032 reported parent PID 41031.
3. Successful `ls` and `date` runs returned status 0. The failure path explicitly used `exit(127)`, so the parent observed 127.

3. Part 2. A Zombie, on Purpose

```
$ ./zombie > /dev/null &
$ sleep 2
$ ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
  40200   40193 Ss   -bash
  44268   40200 S    ./zombie
  44270   44268 Z    [ls] <defunct>
  44271   40200 R+   ps -o pid,ppid,stat,cmd
```

Figure 2.1 – Actual zombie process shown by ps

```
$ ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
  40200   40193 Ss   -bash
  44456   40200 R+   ps -o pid,ppid,stat,cmd
[1]+  Done                    ./zombie > /dev/null
```

Figure 2.2 – Actual process table after waitpid collected the child

3.1. Observe answers

1. `STAT=Z` means the child has terminated but its parent has not yet collected its status.
2. After the parent's 20-second delay, `waitpid()` collected the child and the `<defunct>` entry disappeared.

4. Part 3. Redirection: ls -l > out.txt

```
$ gcc -Wall -Wextra -o redirect redirect.c && ./redirect
open gave descriptor 1
done: the listing is in out.txt
```

Figure 3.1 – Actual redirect program output

```
$ head -3 out.txt
total 68
-rw-r--r-- 1 sofia sofia     0 Oct 19 14:26 out.txt
-rwxr-xr-x 1 sofia sofia 16344 Oct 19 14:26 redirect
```

Figure 3.2 – Actual beginning of the redirected ls output

4.1. Observe answers

1. `open()` returned descriptor 1 because `close(1)` made it the smallest available descriptor.
2. `ls` still wrote to stdout; only the object referenced by descriptor 1 changed from the terminal to `out.txt`.
3. The same method can redirect stderr by closing descriptor 2 before opening the error file.

5. Part 4. A Pipe: ls /etc | wc -l

```
$ gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
183
$ ls /etc | wc -l
183
```

Figure 4.1 – Actual manual and shell pipeline results

```
$ gcc -Wall -Wextra -o pipeline pipeline.c
$ ./pipeline
^C
```

Figure 4.2 – Actual blocked pipeline when the parent kept the write end open

5.1. Observe answers

1. The manually constructed pipeline and the shell pipeline both produced 183.
2. When the parent deliberately kept its copy of the write end open, the `wc` child did not terminate.
3. `wc` was waiting for EOF; EOF cannot be delivered while any process still has a write descriptor referring to that pipe.

6. Conclusion

The programs reproduced core shell behavior using Linux system calls. The work demonstrated process creation and replacement, child-status collection, the zombie state, descriptor-based redirection, and the need to close unused pipe ends so readers can eventually receive EOF.